

PENSAMENTO COMPUTACIONAL



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Aplicações de pensamento computacional

Juliane Adelia Soares

OBJETIVOS DE APRENDIZAGEM

- > Determinar como computadores avaliam fórmulas usando planilhas.
- > Reconhecer a importância de processamento de texto.
- > Descrever o funcionamento de padrões para processamento de texto.

Introdução

Os dispositivos inteligentes, como os computadores, estão cada dia mais presentes em nossas vidas, devido à crescente evolução tecnológica. Esses dispositivos se tornaram onipresentes e são utilizados nas mais diversas áreas, como, por exemplo, na educação e na medicina. Sendo assim, o pensamento computacional tornou-se de grande importância para a sociedade, pois é uma combinação da inteligência humana e dos agentes da computação, visando a resolver problemas complexos. Para isso, podem ser utilizadas fórmulas no contexto de planilhas.

Neste capítulo, você conhecerá as aplicações do pensamento computacional. Além disso, verá como os computadores realizam a avaliação das fórmulas por meio das planilhas. Por fim, você conhecerá a importância do processamento de texto e como funcionam os padrões para tal processamento.

Avaliando fórmulas por meio de planilhas

Os primeiros equipamentos inteligentes foram projetados com uma arquitetura de programa fixa. De acordo com Riley e Hunt (2014), isso significa que as instruções dadas ao computador se incorporavam ao *hardware* real da máquina, tendo apenas os dados processados armazenados na memória. Esses equipamentos não eram reprogramáveis, pois a cada nova instrução que deveria ser enviada, era necessário reiniciá-los para ser possível seguir com a nova sequência.

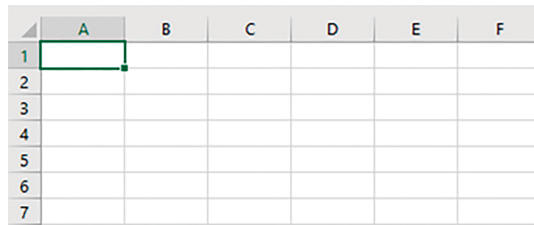
Os computadores atuais se baseiam no conceito de **programa armazenado**, introduzido por John Von Neumann. Ou seja, além dos dados dos programas, as instruções são armazenadas na memória, como uma unidade de armazenamento separada, e são tratadas de forma praticamente idêntica. Esse conceito ficou conhecido como **arquitetura de Von Neumann**. Tal arquitetura permitiu que os dados dos programas pudessem ser alterados por outros programas e que cada um conseguisse alterar as suas próprias instruções. Em suma, a arquitetura de Von Neumann garantiu que os programas pudessem se reprogramar.

Segundo Riley e Hunt (2014), o recurso de reprogramação é chamado de **código automodificável**, e os programas que o possuem são programas que tratam códigos como dados. Um exemplo disso são as **planilhas**, programas interativos que permitem aos usuários organizar, salvar, classificar, gerenciar, processar e exibir dados. Esses dados são armazenados em formato tabular e representam informações em grades bidimensionais de dados, juntamente a fórmulas que os relacionam.

Historicamente, as planilhas são utilizadas para cálculos nas áreas financeiras de um negócio, porém não se limitam a isso. Elas também servem para representar dados científicos, com as fórmulas representando os códigos do programa. A seguir, será apresentado como, por meio das fórmulas, são produzidos os dados e como uma planilha trata o código como dados.

Estrutura de uma planilha

Uma planilha é um documento que armazena dados em uma grade de linhas horizontais e colunas verticais. Em geral, cada uma dessas linhas é rotulada com números, e as colunas, com letras, conforme a Figura 1.



	A	B	C	D	E	F
1						
2						
3						
4						
5						
6						
7						

Figura 1. Estrutura de uma planilha no Microsoft Excel.

A junção entre uma linha e uma coluna é chamada de **célula**, que é rotulada de acordo com a sua localização e tem um nome único. Por exemplo, a célula que está selecionada na Figura 1 é rotulada como A1. Cada célula em uma planilha pode conter números, textos ou fórmulas. As fórmulas derivam novos dados a partir de dados existentes e incluídos em alguma célula da planilha. Se uma célula que está sendo utilizada em alguma fórmula para gerar resultados for alterada, automaticamente, todas as células da planilha que estão relacionadas a ela serão alteradas também, a fim de manter a planilha consistente.



Exemplo

Considere uma planilha que contenha informações de funcionários, em que cada célula é referente a um determinado provento, como salário, horas extras (HE), descontos de vale transporte (VT) e vale refeição (VR). O salário é fixo, e os descontos são realizados sobre o salário-base. Dessa forma, a célula referente às horas extras é alterada mensalmente, de acordo com as horas a mais trabalhadas pelo colaborador. Assim, a cada alteração, a célula que contém o valor total ($\text{salário} - \text{VT} - \text{VR} + \text{HE}$) será atualizada automaticamente, seguindo a alteração desse valor.

É importante destacar que uma célula que contém uma fórmula exibirá os dados calculados pela fórmula, e não a fórmula em si.

Fórmulas

De acordo com o dicionário *on-line* TechTerms, o termo *fórmula* possui diferentes significados: na matemática, é um relacionamento expresso por números e símbolos; na química, define a estrutura atômica de um composto químico; e na computação, é um cálculo executado em uma ou mais variáveis (FORMULA..., 2019).

As fórmulas são o elemento computacional principal em uma planilha, pois definem a computação necessária para a obtenção dos dados de entrada, de modo a produzir dados de saída. Ou seja, por meio de sua aplicação, são obtidos os resultados desejados. Sendo assim, para que esses resultados sejam exibidos de forma correta, faz-se necessário o conhecimento dos elementos que formam uma fórmula, pois todos eles devem seguir uma estrutura interna rígida.

Para que a planilha entenda que está sendo aplicada uma fórmula a uma célula, na maioria dos aplicativos de planilha, é obrigatório que o sinal de igual (=) seja o primeiro digitado antes do restante da fórmula. Riley e Hunt (2014) afirmam que as fórmulas podem consistir apenas nos seguintes elementos: números, operadores, referências de células e funções.

Os **números** são valores aritméticos utilizados para representar quantidades e para a realização de cálculos. Esses são os elementos mais simples utilizados em uma fórmula. A partir de outros elementos, que serão citados a seguir, os números podem ser processados, de modo a produzir números como saída, conforme a Figura 2.

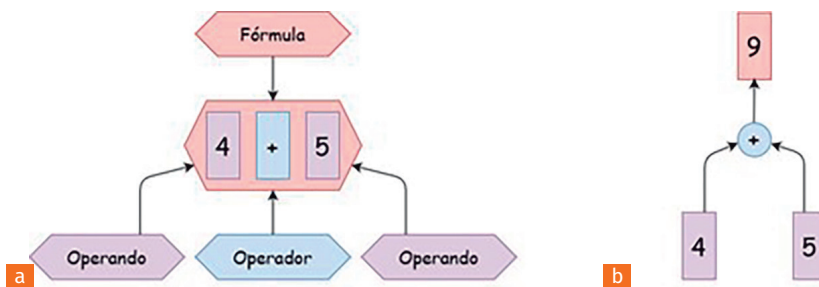


Figura 2. Números em uma fórmula: (a) estrutura de uma fórmula utilizando um operador de adição para processar os números; (b) números processados, em que o operador pega os números 4 e 5 como entrada e produz o número 9 como saída.

Fonte: Adaptada de Riley e Hunt (2014).

Na programação de computadores, os **operadores** são símbolos que representam cálculos. Cada operador aceita uma ou mais entradas, também chamadas de operandos, e gera exatamente um valor de saída. O operando é a entrada do operador, sendo utilizado para gerar o valor de saída. Na Figura 2, existem dois operandos, com o uso do operador de adição (+), gerando um número de saída.

A Microsoft declara que os operadores são responsáveis por especificar o tipo de cálculo que se deseja efetuar nos elementos de uma fórmula (USANDO..., [2021]). O Quadro 1, a seguir, apresenta os operadores referentes aos operadores aritméticos e de comparação.

Quadro 1. Tipos de operadores

Operador aritmético	Significado	Exemplo
+	Adição	$A1 + B1$
-	Subtração	$A1 - B1$
*	Multiplicação	$A1 * B1$
/	Divisão	$A1 / B1$
^	Exponenciação	$A1 ^ B1$
Operador de comparação	Significado	Exemplo
=	Igual a	$A1 = B1$
>	Maior que	$A1 > B1$
<	Menor que	$A1 < B1$
> =	Maior ou igual a	$A1 > = B1$
< =	Menor ou igual a	$A1 < = B1$
< >	Diferente de	$A1 < > B1$

Fonte: Adaptado de Usando... ([2021]).

As fórmulas geralmente contêm vários operadores, portanto, a saída de um operador se torna a entrada ou o operando de outro operador. Por exemplo, observe a fórmula $1 + 4 * 2$, na Figura 3, que possibilita duas interpretações.

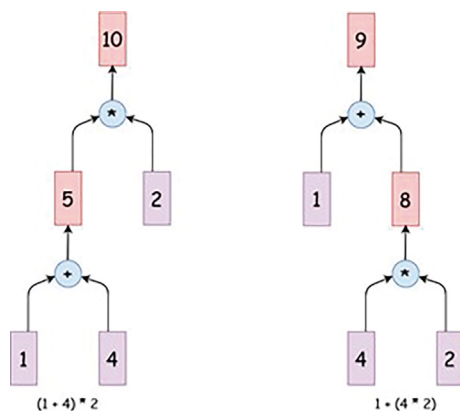


Figura 3. Interpretações da fórmula $1 + 4 * 2$.

Na Figura 3, é possível perceber que, dependendo da interpretação da fórmula, o resultado é diferente. Na primeira interpretação, a saída do operador de adição torna-se a entrada do operador da multiplicação. Já na segunda interpretação, a saída do operador da multiplicação torna-se a entrada do operador da adição.

Mas, então, qual das interpretações está correta? Logicamente, o computador calcula as fórmulas de acordo com uma ordem específica, de modo que o resultado de uma fórmula com vários operadores depende da ordem em que os operandos são aplicados. Segundo Riley e Hunt (2014), como não é possível as fórmulas produzirem resultados diferentes, o computador determinará a ordem de aplicação dos operadores da fórmula. A sequência correta da aplicação do operador pode ser determinada por meio da compreensão dos três atributos distintos de cada operador: aridade, precedência e associatividade. Essa compreensão favorece a interpretação correta de qualquer fórmula com vários operadores.

Segundo Riley e Hunt (2014), a **aridade** de um operador corresponde ao número de entradas solicitadas. Ou seja, o operador unário é o operador de aridade 1, que precisa de apenas uma entrada; o operador binário, de aridade 2, precisa de duas entradas; e o operador ternário, de aridade 3, precisa de

três entradas. Como exemplos de operação binárias, pode-se citar a adição, a subtração, a multiplicação e a divisão. Como operador unário, pode-se citar a negação aritmética.



Saiba mais

A **negação aritmética** se refere à subtração, não considerada apenas quando o primeiro operador for 0. Ou seja, quando se escreve uma frase com “-10”, o operador de negação interpreta como “0-10”.

Como a precedência define a ordem relativa dos operadores, Riley e Hunt (2014) explicam que independentemente da fórmula, o próximo operador de avaliação é o operador de precedência mais alto que ainda não foi avaliado. Nesse caso, os operadores de prioridade mais alta serão avaliados antes dos operadores que possuam prioridade mais baixa. A ordem de precedência dos operadores se dá da seguinte maneira:

1. - (negação);
2. ^ (exponenciação);
3. *, / (multiplicação e divisão);
4. +, - (adição e subtração);
5. <, <=, =, >, >=, <> (menor que, menor ou igual a, igual a, maior que, maior ou igual a, diferente de).

Se uma fórmula contiver dois operadores diferentes com a mesma precedência, uma regra de desempate é aplicada: os operadores serão avaliados da esquerda para a direita.



Fique atento

Os **parênteses** podem ser utilizados para alterar a ordem de avaliação dos operadores. Ou seja, qualquer parte da fórmula entre parênteses pode ser considerada como uma parte independente, estando à frente de todas as demais partes, mesmo com a propriedade de precedência. Por exemplo, na fórmula $(3 + 2) * 2$, sem os parênteses, o resultado seria 7, pois a multiplicação teria prioridade. Contudo, sempre será realizado primeiro o cálculo que estiver entre parênteses, sendo o resultado dessa fórmula: 10.

Se todos os operadores da fórmula forem iguais, a associatividade determina a ordem em que os operadores são aplicados. Os operadores associativos à esquerda são classificados da esquerda para a direita, ao passo que os operadores associativos à direita são classificados da direita para a esquerda. Na maioria dos aplicativos de planilha, os operadores estão à esquerda. No entanto, outras linguagens de programação reconhecem que certos operadores devem ser entendidos como associações essencialmente certas. Por exemplo, a exponenciação é o operador associativo do lado direito de muitas linguagens. Para entender corretamente qualquer fórmula que contenha esse operador, você deve conhecer a associatividade dos operadores.

A Figura 4, a seguir, apresenta a resolução de uma fórmula que envolve mais de um operador, em que a ordem dos operadores está sendo determinada por precedência, associatividade e aridade. É possível observar que, nos casos que envolvem operadores idênticos quanto à precedência, os operadores são avaliados da esquerda para a direita, segundo a associatividade. A saída do operador de adição também é a saída da fórmula, sendo o último operador a ser avaliado.

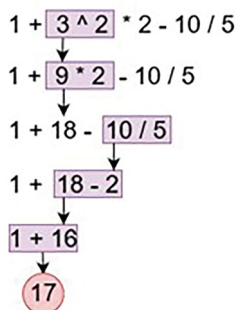


Figura 4. Fórmula com vários operadores.

Fonte: Adaptada de Riley e Hunt (2014).

Conforme o dicionário de computação TechTerms, uma **referência de célula** ou **endereço de célula** é um valor alfanumérico utilizado para identificar uma célula específica em uma planilha. Cada referência de célula contém uma ou mais letras, seguidas de um número, em que as letras identificam a coluna e o número representa a linha (CELL REFERENCE..., 2016).

As células da planilha podem ser consideradas caixas contendo dados valiosos. Como cada caixa tem um nome exclusivo, pode-se acessar os dados fornecendo o nome da caixa (também chamado de referência de célula). Sempre que o computador vê uma referência de célula em uma fórmula, ele recupera os dados da célula referenciada. Em outras palavras, o valor de referência da célula são os dados contidos na célula de referência (RILEY; HUNT, 2014).



Exemplo

Considere que determinadas células têm os seguintes valores: $C1 = 4$; $C2 = 5$; $C3 = 3$; $C4 = 8$. Para saber o valor total dessas quatro células, é inserida uma fórmula na célula C5, fazendo referência às células que contêm os seguintes valores: $= C1 + C2 + C3 + C4$. Após inserida a fórmula, a célula C5 apresentará o valor dessa soma, no caso, 20.

Além de saber o valor da soma, é desejado saber o valor médio do valor total desse cálculo na célula C7. Dessa forma, em vez de buscar uma fórmula em que se adiciona a soma das quatro células novamente para a calcular a média ($= (C1 + C2 + C3 + C4) / 4$), utiliza-se a referência da célula C5, que já contém o resultado da soma, ficando da seguinte maneira: $= C5 / 4$. Assim, a célula C7 apresentará o valor desse cálculo: 5.

A maioria das fórmulas depende de outras células da planilha, sendo assim, a planilha deve determinar a ordem de cálculo das fórmulas. Seguindo a lógica do exemplo citado, a fórmula da célula C7 não poderá ser calculada enquanto a fórmula contida em C5 não tiver realizado o seu cálculo, o que cria uma dependência entre as células. Quem define a ordem em que a fórmula deve ser avaliada, por meio da análise das dependências, é a planilha. Ou seja, uma célula depende da outra quando a fórmula existente apresentar uma referência à outra. Ainda no caso do exemplo apresentado, as células C1, C2, C3 e C4 não têm dependência alguma, pois não possuem referências, apenas valores predefinidos. Já a C5, como possui referências, torna-se dependente das quatro células anteriores. Por fim, a célula C7 é dependente da C5 para que seja calculada e, conseqüentemente, dependente das células C1, C2, C3 e C4.

Quando uma planilha contém muitas fórmulas, ela forma um grafo de dependência de dados, com o intuito de analisar primeiro a dependência de cada fórmula, de modo a determinar qual será avaliada primeiro. Na Figura 5, a seguir, as dependências são mostradas com uma seta apontada de uma

célula para a célula da qual ela depende, sendo possível, assim, determinar a ordem de avaliação das fórmulas.

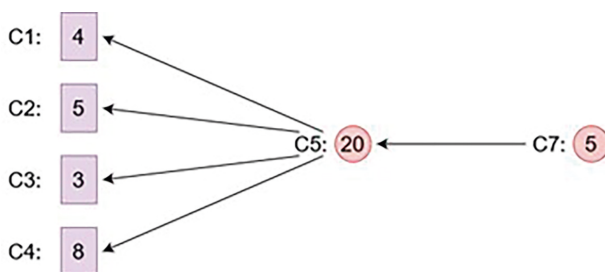


Figura 5. Grafo de dependência.

Fonte: Adaptada de Riley e Hunt (2014).

Sempre que o valor de uma célula que está referenciada em outra for alterado, todas as células que dependem dela serão reavaliadas automaticamente, de modo que a planilha se manterá sempre atualizada.

Em ciências da computação, uma **função** é uma sequência nomeada de instruções que dará um valor de saída quando uma entrada for fornecida. Apesar de algumas funções terem a capacidade de produzir valores mesmo sem um valor de entrada, em quase todos os casos, entradas são fornecidas, as quais serão utilizadas para o cálculo correto do valor de saída. Para usar ou chamar uma função, deve ser inserido o sinal de igual no início da fórmula, o nome da função desejada e a lista de entradas, entre parênteses e separada por vírgula. Por exemplo:

= nomefunção(entrada1, entrada2, entrada3, ...)

Após isso, a função produzirá um valor de saída (RILEY; HUNT, 2014).

Riley e Hunt (2014) complementam que as funções mais comuns de uma planilha incluem: máximo, mínimo, média e soma. Essas funções utilizam intervalos de células como entrada e produzem uma saída, dependendo dos dados existentes no intervalo em questão. O intervalo se refere à uma sub-região retangular de uma planilha, podendo se estender por muitas linhas e colunas. Para definir o intervalo desejado, digita-se primeiro a referência da célula superior e, em seguida, a referência da célula inferior, separando-as por dois pontos (:).

As funções são as seguintes: =máximo, calcula o valor máximo de todas as células no intervalo; =mínimo, calcula o valor mínimo de todas as células no intervalo; =soma, calcula a soma de todas as células do intervalo; =média, calcula o valor médio de todas as células no intervalo. Utilizando o exemplo citado anteriormente, as fórmulas utilizadas podem ser substituídas pelas funções de soma e de média, conforme apresentado na Figura 6.

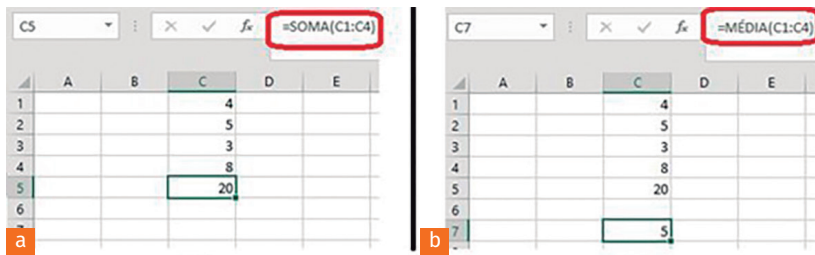


Figura 6. Funções (a) soma e (b) média.

O exemplo da Figura 6 foi elaborado a partir do Microsoft Excel, que é um *software* de planilhas. As funções utilizadas foram: soma, com a função inserida na célula C5 (ver função destacada em vermelho), e média, com a função inserida na célula C7 (ver função destacada em vermelho). Observe que, nas células, só aparecem os resultados das funções.

Processamento de texto

Muitas informações armazenadas nos computadores contêm números, porém a maioria dos dados é armazenado em formato de texto, como, por exemplo, nomes, endereços, *status* do Facebook, entre outros. Todos eles são texto por natureza. Para fazer referência aos dados de texto, é utilizado o termo *string*, que pode ser definido como um pedaço de texto ou uma sequência ordenada de caracteres individuais. Esses caracteres geralmente são letras, mas podem ser sinais de pontuação e caracteres não imprimíveis (i.e., caracteres que não aparecem quando o arquivo é impresso, com a função de demarcação de configurações de formatações), como guias, novas linhas e quebra de páginas (RILEY; HUNT, 2014).

Conforme Riley e Hunt (2014), o número contido em uma *string* se refere ao comprimento desta; caso não possua caracteres, o seu comprimento pode ser zero, ou maior que zero, sendo impossível que tenha um número negativo de caracteres. Os dados das *strings* são indicados por aspas duplas ao redor do texto. A sequência de caracteres entre as aspas duplas é chamada de ***string literal***, porém as aspas não fazem parte da *string*, apenas servem para notificar ao computador que o texto adicionado é uma *string* literal. “Feliz” é um exemplo de uma *string* com cinco caracteres.



Fique atento

Adicionar as sequências de caracteres entre aspas duplas é de extrema importância, caso contrário, o computador não identificará como uma *string* literal. Por exemplo, ao inserir uma data no formato “10/03/52”, se as aspas forem esquecidas, o computador interpretará como uma fórmula de cálculo, já que / é o sinal referente à divisão, e resolverá o cálculo até obter um resultado numérico.

A indexação de caracteres de uma *string* ocorre de modo que o primeiro caractere seja sempre um índice 0, seguindo a ordem de contagem (0, 1, 2, ...). Considerando-se a *string* “Feliz”, o caractere de índice 0 é o “F” e “z” é o caractere no índice 4. O comprimento dessa *string* é cinco, porém o seu maior índice é 4. Posto isso, faz-se necessário destacar que qualquer *string* de comprimento N terá como índice máximo N – 1, ou seja, o intervalo de índices válidos está entre 0 e N-1. Existem algumas operações de *strings* que são utilizadas para o processamento de texto, as quais são apresentadas a seguir.

Indexação (*indexing*)

Como visto, cada um dos caracteres de uma *string* corresponde a um número de índice, iniciando sempre em 0. Tanto os espaços em branco quanto qualquer símbolo ou sinal de pontuação têm números de índice associado. Por exemplo, em “Feliz Natal!”, o “F” é o índice 0, e “!”, o índice 11 (TAGLIAFERRI, 2016).

O índice é utilizado para analisar uma *string*. A indexação é uma operação de índice, por meio da qual é possível acessar os caracteres de uma *string* individualmente, utilizando o valor numérico referente ao índice. Para selecionar um dos caracteres em uma *string*, deve-se colocar o número do índice desejado entre colchetes, após a *string* literal, conforme a expressão apresentada na Figura 7.

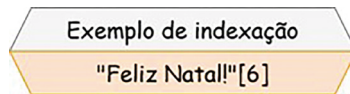


Figura 7. Exemplo de indexação.

Dessa forma, será retornada a letra “N”, que é o sétimo caractere que se encontra no índice de posição 6. Se o índice colocado entre colchetes não existir, se fosse colocado o número 12, por exemplo, a planilha retornaria um erro, pois, nesse caso, o valor de índice válido encontra-se no intervalo de 0 a 11.

Comprimento (*length*)

É possível obter o comprimento de uma *string* literal digitando a operação “.length” após o final da *string*, conforme a Figura 8.

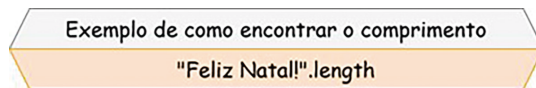


Figura 8. Exemplo da operação de comprimento.

A expressão apresentada na Figura 8 retornará o número 12, que é o número de caracteres da *string* literal. Ou seja, como resultado, o código "Feliz Natal!".length será substituído pelo valor do comprimento, nesse caso, o número 12.

Concatenação (*concatenation*)

A concatenação é outra operação comum de processamento de *strings*. Essa operação pega *strings* e as une, de modo a formar uma *string* de saída. É utilizado o símbolo de mais (+) para concatenar as *strings* em questão, conforme a Figura 9.



Figura 9. Exemplo de concatenação.

A Figura 9 mostra a concatenação entre “Feliz” e “Natal” com uma *string* de indicação de espaço entre elas produzindo a *string* de saída “Feliz Natal!”.

Nomenclatura (*naming*)

As variáveis são utilizadas para se referir a dados, sendo associadas aos dados por meio de uma operação de associação de nomes. Isso é feito por meio do símbolo de seta para a esquerda (←), que, à sua esquerda, deve conter um nome de variáveis, e à sua direita, um valor. A Figura 10, a seguir, apresenta um exemplo de variável de *string*.

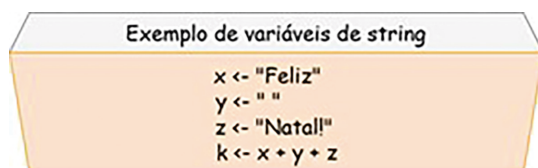


Figura 10. Exemplo de variável de *string*.

Na Figura 10, foram utilizadas variáveis de *strings* para nomear as literais de *strings*. O computador é instruído a vincular o nome x a *string* literal “Feliz”, o nome y ao espaço que deve existir na *string* final, o nome z ao “Natal!” e o nome k a *string* “Feliz Natal!”, que é a *string* formada pela concatenação das *strings* referidas nas variáveis x, y e z. Pode ser difícil distinguir entre o nome

de uma *string* e a própria *string*, devido aos caracteres utilizados para ambos. Cientistas da computação afirmam que os nomes ocorrem em um *namespace* (espaço de nome), nesse caso, as variáveis *x*, *y*, *z* e *k*, ao passo que as *strings* “Feliz”, “ “, “Natal!” ocorrem em *value space* (espaço de valor) do programa.

Substring

A indexação permite obter um caractere individual de uma *string*, porém a *substring* permite obter uma subsequência de uma *string*. Ou seja, se os índices do primeiro e do último caractere que se deseja extrair da *string* forem conhecidos, é possível selecionar apenas a sequência desejada.

A Figura 11, a seguir, apresenta um exemplo de uma *string* que está sendo aplicada na variável *x*, sendo “Feliz Natal!”. A orientação enviada ao computador é que seja dada a sequência iniciando no índice 1 e terminando no índice 3. Contudo, para isso, faz-se necessário selecionar até o índice 4 para obter o caractere que corresponde ao índice 3, pois o número 4 denota o índice do primeiro caractere que não está sendo incluído na saída. Ou seja, o último caractere incluído na subsequência produzida pela operação de *substring* está no índice 3. Quando o código é executado, a variável *y* retorna a *string* “eli”, que corresponde ao intervalo selecionado.

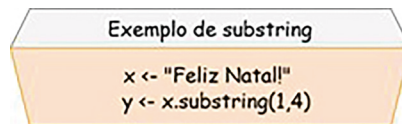


Figura 11. Exemplo *substring*.

Pesquisando (*searching*)

Essa operação de processamento de *string* serve para identificar em qual posição de índice determinado caractere se encontra, por meio da função *indexOf*. A Figura 12, a seguir, apresenta um exemplo em que a *string* em questão é “Feliz Natal!” e se deseja saber em qual posição o caractere “z” se encontra. Quando executado o código “Feliz Natal!”.*indexOf*(“z”), ele é substituído pelo índice 4, que é onde se encontra o caractere “z” na *string* citada. Se fosse buscado um caractere inexistente na *string*, como o “o”, por exemplo, o retorno obtido seria o número -1, que é um índice inválido, de modo a informar que o caractere não se encontra na *string* literal.

Pesquisando com `indexOf`
`x <- "Feliz Natal!".indexOf("z")`

Figura 12. Exemplo de pesquisa.

Padrões para processamento de texto

Os padrões são considerados técnicas úteis no processamento de dados textuais, pois definem propriedades que *strings* podem possuir ou não. Essa é uma forma de definir se uma *string* é membro de uma família definida pelo padrão ou não.

O Quadro 2, a seguir, apresenta o conceito de um padrão. As *strings* contidas na primeira coluna têm um padrão conhecido, como, por exemplo, “123-45-6789”, que segue um padrão de números da Previdência Social. Ou seja, o Seguro Social possui 3 dígitos seguidos por um traço (-), mais 2 dígitos e um traço (-) e mais 4 dígitos finais. *Strings* nesse padrão serão facilmente reconhecidas como do Seguro Social, assim como as que estão fora do padrão não o serão.

Quadro 2. Padrões textuais

Padrões textuais comuns	
<i>String</i> literal	Família padrão
“123.456.789-00”	Número do CPF
“(51) 99900-1234”	Número de telefone
“beatles@sub.edu”	Endereço de <i>e-mail</i>
“13/04/1963”	Data
XOXO	Abraços e beijos
“1234-1234-1234-1234”	Número do cartão de crédito
“KPI-6473”	Placa de automóvel (formato brasileiro)
“QOT-4J33”	Placa de automóvel (formato Mercosul)

Fonte: Adaptado de Riley e Hunt (2014).

No âmbito da programação de computadores, padrões de *strings* são conhecidos como expressões regulares, que servem para definir a correspondência de padrões. Esse tipo de técnica serve para processar dados textuais em grandes escalas, como grandes bancos de dados. Para escrever um padrão válido e entender o significado de qualquer padrão escrito, existem algumas regras, apresentadas no Quadro 3.

Quadro 3. Regras para escritas de padrões

Regras básicas para a escrita de padrões
Qualquer caractere único (exceto alguns caracteres especiais) é um padrão. O caractere representa a si mesmo.
O símbolo de ponto final (.) é um padrão. O caractere representa qualquer caractere único. O período é especial.
Se A e B são ambos padrões, então são: ■ A B: O sequenciamento representa o padrão A seguido pelo padrão B ■ A B: Alternância representa o padrão A ou o padrão B ■ (A): O agrupamento deixa de lado o padrão A como um grupo. Os parênteses são especiais.

Fonte: Adaptado de Riley e Hunt (2014).

Para o padrão da *string* abraços e beijos, por exemplo, existe apenas uma *string* literal que corresponde ao padrão (XOXO) e é definida como um padrão fixo. É possível provar que esse é um padrão, pois segue algumas das regras básicas que são aplicadas em elementos padrões, especificamente a 1ª e a 3ª. O único caractere X é um padrão, sendo O (o único caractere) também um padrão, o que prova que seguem a regra 1 do Quadro 3. Dessa forma, XO segue a 3ª regra, pois é uma sequência do padrão X juntamente ao padrão O. A *string* completa, XOXO, também é um padrão de acordo com a 3ª regra, pois é uma sequência de ambos os padrões XO e XO. Esse é um sequenciamento de subpadrões.

Os números do CPF também seguem um padrão, podendo ser entendidos seguindo a seguinte sequência: três dígitos, ponto, três dígitos, ponto, três dígitos, um traço, dois dígitos. Podem ser utilizados quaisquer números, desde que estejam dentro dos 10 dígitos padrão (0, 1, 2, 3, 4, 5, 6, 7, 8, 9), fazendo uso da alternância para escrever o padrão que descreve um único dígito. Esse padrão utiliza agrupamentos, com o intuito de isolar o subpadrão de dígito de outras partes que contenham um dígito (Figura 13).

Padrão de CPF						
(0 1 2 3 4 5 6 7 8 9)	(0 1 2 3 4 5 6 7 8 9)	(0 1 2 3 4 5 6 7 8 9)	.	(0 1 2 3 4 5 6 7 8 9)	(0 1 2 3 4 5 6 7 8 9)	(0 1 2 3 4 5 6 7 8 9)
(0 1 2 3 4 5 6 7 8 9)	(0 1 2 3 4 5 6 7 8 9)	(0 1 2 3 4 5 6 7 8 9)	-	(0 1 2 3 4 5 6 7 8 9)	(0 1 2 3 4 5 6 7 8 9)	

Figura 13. Padrão de CPF.

Riley e Hunt (2014, p. 248) afirmam que o padrão é algo como um número do CPF, ou seja, uma sequência composta por: um dígito; um dígito; um dígito; um ponto; um dígito; um dígito; um dígito, um ponto; um dígito; um dígito; um dígito; um traço; e, finalmente, um dígito e mais um dígito. Esses padrões são longos e difíceis de ler, o que levou ao desenvolvimento de melhores técnicas para a escrita de padrões, como as regras de repetição e as regras de classe de caractere, descritas a seguir.

Regras de repetição

Existem diversas regras adicionais para a escrita de padrões, as quais permitem que desenvolvedores controlem quantas vezes um elemento pode ser repetido. As repetições auxiliam a criar padrões mais consistentes. Por exemplo, se um número de CPF possui um padrão de 3 dígitos, 3 dígitos, 3 dígitos e, por fim, 2 dígitos, então é possível estabelecer um número exato de repetições usando a regra 4d para gerar um padrão. O Quadro 4, a seguir, demonstra como escrever o padrão de número do CPF utilizando regras de repetição.

Quadro 4. Regras de repetição

Regras de repetição

Se A é um padrão, então:

- A^* : denota zero ou mais repetições de A . O^* é especial.
- A^+ : Denota uma ou mais repetições de A . O^+ é especial.
- $A?$: Denota zero ou uma ocorrência de A . $O?$ é especial.
- $A\{m\}$: Denota exatamente m repetições de A . As chaves são especiais.
- $A\{m, n\}$: Denota pelo menos m e não mais que n repetições de A .
- $A\{m, \}$: Denota pelo menos m repetições de A .

Fonte: Adaptado de Riley e Hunt (2014).

Inicialmente, com a regra 4d, define-se que um número de CPF deve começar com 3 dígitos, seguido da adição de um ponto (.). Então, usa-se a regra 4d novamente para declarar os 3 dígitos seguintes, mais um ponto (.). Depois, usa-se novamente a regra 4d para declarar mais 3 dígitos e um travessão (—) e, então, os 2 dígitos finais. É importante observar que um caractere de repetição é aplicado apenas ao padrão imediatamente anterior, sempre que *, +, ?, {M} ou {m,n} ocorrem. No caso do CPF, os grupos de dígitos aparecem imediatamente antes de cada um dos controles de repetição, permitindo que apenas os dígitos possam ser repetidos, e não os travessões (—).

Outro exemplo pode ser a expansão do padrão citado de abraços e beijos para incluir *strings* que repitam o padrão XO. Ou seja, cada uma das *strings* que correspondem ao padrão expandido de abraços e beijos: “XO”, “XOXO”, “XOXOXO”, “XOXOXOXO” etc. Assim, o padrão (XO)+ define uma família expandida de abraços e beijos de *strings*, já que o símbolo de “+” mostra as repetições. O uso de parentes em (XO) é de suma importância para a repetição do termo ocorrer de forma correta, pois, sem os parênteses, o padrão buscaria toda e qualquer *string* que começasse com XO.

Regras de classe de caractere

Classe de caractere é o mesmo que conjunto de caracteres, como, por exemplo, o termo *dígito*, que se refere a uma classe de caracteres, sendo ela 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9. A regra de escrita de padrões possibilita a definição de diversas classes de caracteres, com padrões de uso comuns e classes de caracteres nomeadas. O Quadro 5, a seguir, apresenta as classes de caracteres, em que cada sub-regra nomeia uma classe específica.

Quadro 5. Regras de classe de caractere

Regras de classe de caracteres
Os colchetes denotam uma classe de caractere. Cada caractere entre colchetes é um membro da classe de caractere. Um traço em uma classe de caracteres denota uma variedade de caracteres. ■ \d: Denota qualquer dígito único; ■ \D: indica qualquer caractere que não seja um dígito; ■ \w: Denota qualquer caractere de palavra (a-z, A-Z, 0-9); ■ \W: denota qualquer caractere diferente de palavra; ■ \s: indica qualquer caractere de espaço em branco (espaço, tabulação, avanço de linha); ■ \S: Denota qualquer caractere que não seja um espaço em branco.

Fonte: Adaptado de Riley e Hunt (2014).

A classe de caracteres de palavras (`\w`) busca qualquer caractere que possa aparecer em palavras, como os alfabéticos ou os dígitos. Dos que não podem aparecer em palavras, pode-se citar os símbolos, como `?`, `@`, `:`, `#`, entre outros, que são adicionados à classe de caracteres não palavra (`\W`).

A classe de caracteres de espaço em branco serve para separar as palavras digitadas por um processador de texto. O caractere de espaço no teclado é membro dessa classe, mas os caracteres de tabulações e alimentações de linha, como os resultantes da tecla ENTER, também fazem parte dela. Com a classe de caractere, pode-se escrever o padrão de CPF de forma consistente, cujo padrão final é mostrado na Figura 14.

Padrão de CPF com Repetições e Classes de Caractere
<code>\d{3}.\d{3}.\d{3}-\d{2}</code>

Figura 14. Padrão de CPF com repetições e classes de caractere.

Nesse caso, um dígito pode ser expresso por meio do uso de uma classe de caractere com qualquer das opções `[0123456789]`; ou seja, quaisquer dos dígitos que estiverem entre colchetes. Já o padrão `[0-9]` representa qualquer caractere no intervalo de “0 a 9”, e “`\ d`” representa qualquer dígito. Sendo assim, pode-se utilizar a classe de caracteres de dígito, “`\ d`”, para representar um único dígito ao padrão de CPF.

Ao longo deste capítulo, foram discutidas as fórmulas utilizadas em planilhas e o modo como os computadores as avaliam. Além disso, foram observadas a importância das operações que processam os textos e a maneira que os padrões de processamento de textos funcionam. Como visto, as planilhas são utilizadas para substituir códigos por dados. Muitos dos dados gerados pelos computadores são numéricos, porém, em sua grande maioria, são armazenados em forma de texto. Diante disso, existem operações que são realizadas pelos computadores com o intuito de processar esses dados textuais. Além das operações, existem padrões para processamento de texto, que são definidos por algumas regras.

Referências

CELL REFERENCE Definition. In: TECHTERMS: The Computer Dictionary. Minneapolis: Sharpened Productions, 2016. Disponível em: https://techterms.com/definition/cell_reference. Acesso em: 9 jan. 2021.

FORMULA Definition. In: TECHTERMS: The Computer Dictionary. Minneapolis: Sharpened Productions, 2019. Disponível em: <https://techterms.com/definition/formula>. Acesso em: 9 jan. 2021.

RILEY, D. D.; HUNT, K. A. *Computational thinking for the modern problem solver*. Boca Raton: CRC Press, 2014. 408 p.

TAGLIAFERRI, L. How To Index and Slice Strings in Python 3. *DigitalOcean*, New York, 28 Sep. 2016. Disponível em: <https://www.digitalocean.com/community/tutorials/how-to-index-and-slice-strings-in-python-3>. Acesso em: 9 jan. 2021.

USANDO Operadores de Cálculo em Fórmulas do Excel. *Microsoft*, Redmond, [2021]. Disponível em: <https://support.microsoft.com/pt-br/office/usando-operadores-de-c%C3%A1lculo-em-f%C3%B3rmulas-do-excel-78be92ad-563c-4d62-b081-ae6da5c-2ca69>. Acesso em: 9 jan. 2021.

Leitura recomendada

JARGAS, A. M. *Expressões regulares: uma abordagem divertida*. 5. ed. São Paulo: Novatec, 2016. 247 p.



Fique atento

Os links para sites da web fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integridade das informações referidas em tais links.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS